

Dictionaries and Sets

Eunbyung Park
Yonsei University



Textbook Chapter 9

Dictionaries and Sets

Disclaimer: the following slides are based on the lecture slides provided by the textbook author, Copyright 2013 John Wiley and Sons. Material added/revised by the course instructor Copyright 2024 Yonsei University.

These course materials have been produced and published for the purpose of teaching Yonsei University students, so they cannot be used for any other purposes nor be shared with others. The legal responsibility for the violation lies with the actor him/herself.

Motivation

- The vast amounts of **data** being stored today could not be effectively utilized without being **organized** in some way.
- Databases do not simply store large amounts of data. Rather, they provide a complete database management system (DBMS) used in conjunction with stored data to allow the data to be associated and accessed in various ways.
- Some of the concerns of database management systems include:
 - **integrity** of data—that is, data that is accessible and properly organized;
 - **security** in the control of who has what level of access;
 - **concurrency issues** when multiple users are accessing the same data.
- **Data mining** is a relatively new field in which database management systems are used in conjunction with methods from statistics and artificial intelligence to extract patterns from very large amounts of data.

Areas	Data Mining Application
Business	Customer relationship management (find, attract, retain customers)
Genetics	DNA analysis (understand individual variations in human DNA in relation to disease susceptibility)
National Security	Behavioral patterns (identify possible terrorist activities)
Information Retrieval	Ex.: music information retrieval to discover users' behavioral patterns in music retrieval

Data Mining Applications

Motivation

- In this lecture, we're studying how we can efficiently organize the data that our Python program maintains in memory.
- To facilitate ease of access (for our own code).

Motivational example: storing student info

- Until now, we used the **list index** to **organize** information:

names = ['Jiin', 'Jinoo', 'Ana']

grades = ['A+', 'B', 'A0']

courses = ['2100', '2102', '3100']

- Student names **kept separately** from their data (grades, courses).
- Each list must have **same length**.
- Information of a student must be stored **consistently** at **same index** across lists.
 - Examples:
 - Jiin: index 0
 - Ana: index 2

Looking up student info

```
def getGrade(student, name_list, grade_list, course_list):
```

```
    i = name_list.index(student)
```

Determine list index of student.

```
    grade = grade_list[i]
```

```
    course = course_list[i]
```

```
    return (grade, course)
```

```
x = getGrade('Jinoo', names, grades, courses)
```

- **Messy** if a lot of information to keep track of.
- Must maintain **many lists** and pass around as arguments.
- Must **index using integers** (easy to make mistakes).
- Remember to change multiple lists (easy to make mistakes).

A Cleaner Solution: Dictionaries

- Can **directly index item of interest** (not necessarily integer)

List:

0	Element 1
1	Element 2
2	Element 3
3	Element 4
...	...

integer index
element

Dictionary:

Key 0	Value 1
Key 1	Value 2
Key 2	Value 3
Key 3	Value 4
...	...

custom index
element

- A **list index** is always of type **integer**. ☹️
- The **index** of a **dictionary** can be of **any type**. 😊

Python Dictionaries

- A dictionary stores **key-value pairs**:
 - <key, value>

Dictionary:

'Jiin'	'A+'
'Jinoo'	'B+'
'Ana'	'A0'

custom
index
(key)

element
(value)

my_dict = {} empty
dictionary

grades = {'Jiin': 'A+', 'Jinoo': 'B', 'Ana': 'A0'}

key 0
value 0

Dictionary Lookup

A dictionary lookup:

- **Looks up** the **key**
- **Returns** the **value** associated with the key
- Raises exception **KeyError** if key is **not found**

Dictionary:

'Jiin'	'A+'
'Jinoo'	'B+'
'Ana'	'A0'

key

value

```
grades = {'Jiin': 'A+', 'Jinoo': 'B', 'Ana': 'A0'}
```

```
grades['Jiin'] → 'A+'
```

```
grades['Carl'] → KeyError exception
```

Dictionary Add/Test/Delete Operations

```
grades = {'Jiin': 'A+', 'Jinoo': 'B', 'Ana': 'A0'}
```

- **Add** an entry:

```
grades['Carl'] = 'A+'
```

- **Test** if key in dictionary:

```
'Jinoo' in grades → evaluates to True
```

```
'Peter' in grades → evaluates to False
```

- **Delete** an entry from the dictionary:

```
del grades['Jiin']
```

Dictionary:

'Jiin'	'A+'
'Jinoo'	'B+'
'Ana'	'A0'

Iteration on Dictionary Keys

```
grades = {'Jiin':'A+', 'Jinoo':'B', 'Ana':'A0'}
```

'Jiin'	'A+'
'Jinoo'	'B+'
'Ana'	'A0'

- **Iterate** across the **keys** of a dictionary:
- The **keys()** method returns a **dict_keys** object containing all keys in **insertion order**.
 - Insertion order only preserved since Python 3.7.
- dict_keys objects are usable for iteration (loops)

```
for person in grades.keys():  
    print(person)
```

insertion order

Output (in insertion order):

Jiin
Jinoo
Ana

Code Layout for Large Dictionary Literals

```
grades = {  
    'Jiin': 'A+',  
    'Jinoo': 'B',  
    'Ana': 'A0',  
    'Peter': 'B+',  
    'Yousun': 'A+',  
    'Minyoung': 'A+',  
    'John': 'A+',  
    'Timothy': 'A+',  
}
```

This code layout works well for dictionary literals with many entries.

- The final comma is redundant.
- But it saves you one error if you append a new entry in a hurry. 😊

- Dictionary literals with **many entries** won't fit on a single line in your editor.
- A better code layout in this case is the vertical layout that puts one entry per line.

Iteration on Dictionary Values

```
grades = {  
    'Jiin': 'A+',  
    'Jinoo': 'B',  
    'Ana': 'A0',  
}
```

'Jiin'	'A+'
'Jinoo'	'B+'
'Ana'	'A0'

- **Iterate** across the **values** of a dictionary:
- The **values()** method returns a **dict_values** object containing all values, in the insertion order of the key of each value.
 - dict_values objects can be used for iteration (loops):

```
for grade in grades.values():  
    print(grade)
```

Output:

A+
B+
A0

Dictionary Keys are Unique

A dictionary is an **associative data structure**.

- Associates a value with a key.

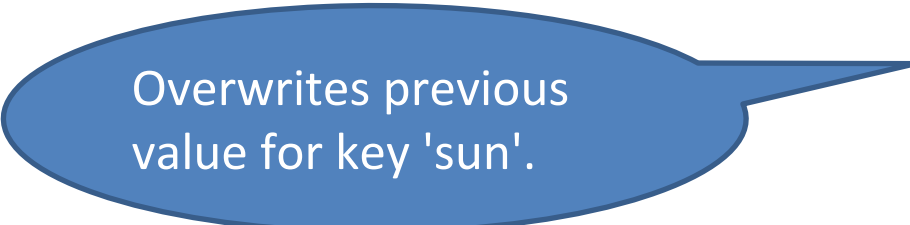
Keys of a dictionary are **unique**.

- Assigning a new value to a key **overwrites** the previous value.
- See example to the right.

```
daily_temps = {  
    'sun': 28.8,  
    'mon': 30.2,  
    'tue': 37.3,  
    'wed': 32.8,  
    'thu': 33.2,  
    'fri': 25.6,  
    'sat': 24.0,  
}
```

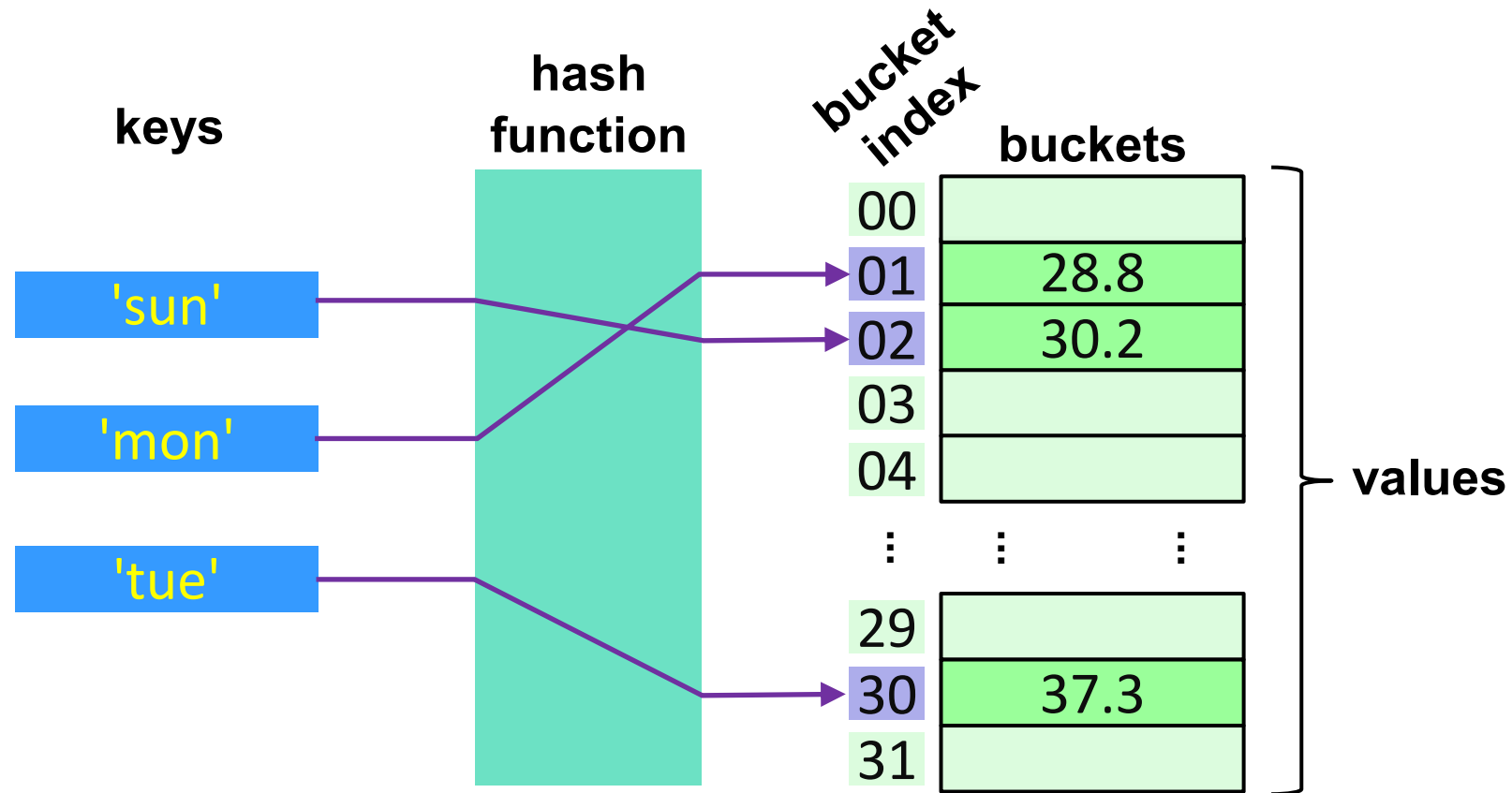
```
print(daily_temps['sun']) # 28.8  
x = -10
```

```
daily_temps['sun'] = x  
print(daily_temps['sun']) # -10
```



Overwrites previous
value for key 'sun'.

Hashing:



- Each value is stored in a **bucket**.
- The **hash function** determines the **bucket index** of a value from its key.
 - Step 1: convert the key to an **integer x**.
 - Step 2: use the integer x as the **bucket index**.

- The hash function is **deterministic**.
 - Locations are **not** chosen randomly.
 - During one run of your program, the same key will always produce the same bucket index.
 - But keys are "salted" with a random number specific to each program run (to guard against denial-of-service attacks).

Keys, Values and Hashing

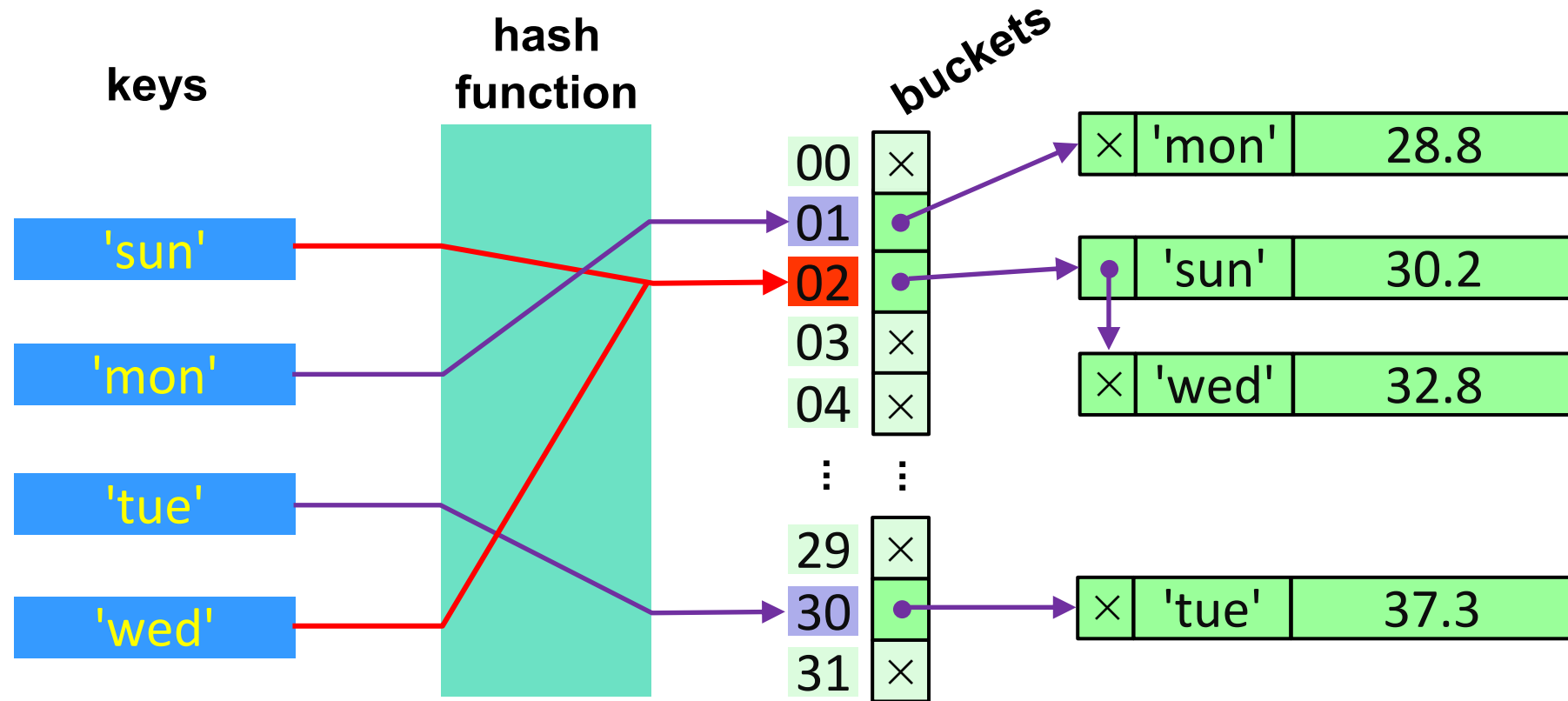
- **Keys** are **un-ordered** (conceptually).
 - Must be **unique** (see previous slide)
 - Must be **hashable**, which means the hash key type must be **immutable**.
 - Ex.: tuple, string, int, float ok. But lists are not allowed.
 - Logically, there is no first element, second element, etc. in a dictionary.
- **Values:**
 - can have **duplicates**
 - can be any type (**immutable** and **mutable**)
 - dictionary value can be a list, or even another dictionary.
- Python's built-in hash() function computes an integer value (**hash**) for every hashable object (see next slide).

The Built-in Hash() Function

```
>>> hash('sun') → 4775822037878443550
>>> hash(1) → 1
>>> hash(2) → 2
>>> hash(2.0) → 2
>>> hash(2.1) → 230584300921369602
>>> hash('hello') → -8369422590941704495
>>> hash(('mytuple', 1)) → 7120811234780347391
>>> hash([]) → TypeError: unhashable type: 'list'
>>> hash(2.1) == hash(230584300921369602) → True # hash collision
```

- The inner workings of the hash() function are an implementation detail of the Python interpreter and usually of no concern to the programmer.
 - But they are useful to understand how the hash function works.
- A computed hash may be too large and needs to be truncated to fit the size of the dictionary.
 - Ex.: assuming a dictionary with 32 slots:
 - $4775822037878443550 \% 32 \rightarrow 30$
- If the hash function generates the same hash for two keys, the hash function has a **collision**.

Hash Collisions



- One way to resolve collisions in a hash table is to use a linked list to track key-value pairs that share the same keys.
 - See above example that has a collision of the keys 'sun' and 'wed', both mapping to bucket index 02.
- Python uses a different approach, called open addressing.
- Intuitively, a collision incurs performance overhead.

Hash Key Types

- Although strings are often used as key values, **any immutable type** may be used as a **key**, even a **tuple**.
- Example:** the temperature (value) for a specific month-and-year (key) is retrieved by:

```
tmps = {  
    ('Jan', 2004) : -15,  
    ('June', 2015) : 22.5,  
    ('Feb', 2008) : -14.3,  
    ('Oct', 2007) : 16.8  
}
```

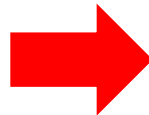
buckets

16.8
-15
22.5
-14.3

tmps[('June', 2015)]

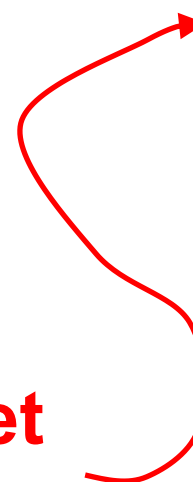


key



**Hash
Function**

**bucket
index**



Hash Value Types

- **Values:**
 - can have **duplicates**
 - can be any type (**immutable** and **mutable**)
 - dictionary value can be a list, or even another dictionary.
- **Example:** temperature values for a specific month:

```
tmpr = {  
    'Jan': [-1, 0, -10],  
    'Feb': [-14.3, -12.0, -10.0],  
    'Oct': [22, 21, 19],  
}
```

Comparing Lists and Dictionaries

List:

- **Ordered** sequence of elements.
- Elements looked up by integer index.
- Index type is **integer**.
- Map from integer (index) to element.

Dictionary:

- Associates a **value** with each **key**
- Values are **un-ordered**.
- Keys are **un-ordered**.
- Keys can be any **immutable type**.
- Map from key to value
 - See example on next slide

```
dict_temps = {'sun': 23, 'mon': 24, 'tue': 22,  
              'wed': 28, 'thu': 29, 'fri': 30, 'sat': 27}
```

```
list_temps = [['sun', 23], ['mon', 24], ['tue', 22],  
              ['wed', 28], ['thu', 29], ['fri', 30],  
              ['sat', 27]]
```

- A dictionary is a **map** from **keys** to **values**:
 - map: $x \in \text{key-type} \longrightarrow y \in \text{value-type}$
- Your program needs only **one operation** to look up the value for a key 😊:
 - Example: `dict_temps['thu']`
- With a list, your program needs **N operations** for a lookup,
if you don't know the index (ex: for data from a file):
 - Example:

```
for k in list_temps:  
    if k[0] == 'thu':
```

To find 'thu', we need
to go over the entire
list (worst-case)

Dictionary Operations (Summary)

Operation	Result
<code>dict()</code>	Creates a new, empty dictionary.
<code>dict(s)</code>	Creates a new dictionary with key-value pairs taken from a sequence s, ex.: <pre>fruit_data = [['apples', .66], ['pear', 0.87], ['mangos', 0.56]] fruit_prices = dict(fruit_data)</pre>
<code>len(d)</code>	Length (number of key-value pairs) of dictionary d.
<code>d[key] = value</code>	Set the value associated with key to value. If the key did not exist yet, a new key-value pair is added. Otherwise, the existing key-value pair is replaced.
<code>del k[key]</code>	Remove key and associated value from dictionary d.
<code>key in d</code>	True if key exists in dictionary d, otherwise False.

Self-Test Questions

1. A dictionary type in Python is an associative data structure that is accessed by a _____ rather than an index value.
2. Associative data structures such as the dictionary type in Python are useful for,
 - (a) accessing elements more intuitively than by use of an indexed data structure
 - (b) maintaining elements in a particular order
3. Which of the following types can be used as a key in Python dictionaries?
 - (a) strings
 - (b) lists
 - (c) tuples
 - (d) numerical values
4. Which of the following is a syntactically correct sequence, `s`, for dynamically creating a dictionary using `dict(s)`.
 - (a) `s = [[1: 'one'], [2: 'two'], [3: 'three']]`
 - (b) `s = [[1, 'one'], [2, 'two'], [3, 'three']]`
 - (c) `s = {1: 'one', 2: 'two', 3: 'three'}`
5. For dictionary `d = {'apples':0.66, 'pears':1.25, 'bananas':0.49}`, which of the following correctly updates the price of bananas.
 - (a) `d[2] = 0.52`
 - (b) `d[0.49] = 0.52`
 - (c) `d['bananas'] = 0.52`

ANSWERS:

The Set Data Type

A **set** is a mutable data type with nonduplicate, unordered values, providing the usual mathematical set operations.

Set operator		Set A = {1,2,3}	Set B = {3,4,5,6}	
membership		<code>1 in A</code>	True	<i>True if 1 is a member of set A</i>
add		<code>A.add(4)</code>	{1,2,3,4}	<i>Adds new member to set</i>
remove		<code>A.remove(2)</code>	{1,3}	<i>Removes member from set</i>
union		<code>A B</code>	{1,2,3,4,5,6}	<i>Set of elements in either set A or set B</i>
intersection		<code>A & B</code>	{3}	<i>Set of elements in both set A and set B</i>
difference		<code>A - B</code>	{1,2}	<i>Set of elements in set A, but not set B</i>
symmetric difference		<code>A ^ B</code>	{1,2,4,5,6}	<i>Set of elements in set A or set B, but not both</i>
size		<code>len(A)</code>	3	<i>Number of elements in set A (general sequence operation)</i>